



Department of Computer Science and Engineering
Premier University

CSE 452: Neural Network & Fuzzy Logic Laboratory

**Title: Text Classification of Bangla News Articles Using Recurrent
Neural Networks (RNN)**

Submission Date: 23.11.2025

Submitted by:

Name	ID
Shuvra Roy	0222210005101093
Arnab Shikder	0222210005101098
Mohammad Hafizur Rahman Sakib	0222210005101118

Submitted to:

MD Tamim Hossain
Lecturer, Department of CSE
Premier University
Chittagong

Remarks

Text Classification of Bangla News Articles Using Recurrent Neural Networks (RNN)

Abstract

This project is about classifying Bangla news by using Recurrent Neural Network (RNN). Our main goal is to separate the news into five types: economy, entertainment, international, sports, and state. We used a dataset of 2,500 news from Prothom Alo, Jugantor and some other sites. This work is important because Bangla news sorting is little hard since resources are not enough.

At first, we cleaned the text like removing extra space and non-Bangla letters. After that, we tokenized the words and made a list of 10,000 common words. Then we changed the text into numbers and padded every text to 500 words. We divided the data into train (2,000) and test (500). After that, we made different models like RNN, LSTM and GRU. We also tested some changes like using 1 to 3 layers, using bidirectional model, using dropout (0.3 to 0.5), and using different optimizers like Adam and SGD. We trained all models for 6 epochs and checked the loss and accuracy.

The simple RNN got only around 21% accuracy. When we added two layers and bidirectional to LSTM, the accuracy increased to 69%. Our best accuracy came from Bi-GRU which gave 73%. Still, there was some confusion between entertainment and sports, and also between international and state. Bidirectional worked well because it can read the text forward and backward. More layers helped the model learn better, but without dropout it can overfit.

Overall, RNN-based models work good for Bangla news classification. Using bidirectional and LSTM/GRU gives better result because they understand word order nicely. In the future, we can use pre-trained word vectors or collect more data to improve the accuracy. This project shows that we can tune simple models to get better performance even in low-resource languages like Bangla.

Keywords: Bangla news classification, Recurrent Neural Network (RNN), LSTM, GRU, bidirectional RNN, text preprocessing, low-resource language, deep learning for Bangla NLP, hyperparameter tuning, dropout, optimizer comparison

1 Introduction

In today's time, we see news everywhere, and people read news on phone or computer every day. But putting the news into groups like economy, sports, entertainment takes time. For English language, computer can do this job easily using machine learning. But for Bangla, it is harder because we don't have enough tools or datasets. Bangla is spoken by more than 250 million people, but still computer cannot understand it well.

In this project, we try to classify Bangla news using Recurrent Neural Network (RNN). RNN works good for text because it remembers the word order, like how a story is written.

Our dataset has total 2,500 Bangla news articles. We collected them from Prothom Alo, Jugantor, Ittefaq, Bdnews24, Kaler Kantho, Bangla Tribune and Samakal. The dataset is in a CSV file with two columns: "text" for the news and "label" for the category. There are five categories: economy, entertainment, international, sports and state. Each category has 500 news, so the data is balanced. For example, an international news may say: "ভারতের কেন্দ্রীয় শহর উন্নয়ন মন্ত্রী এম ভেঙ্কইয়া নাইডু বলেছেন..." and the label is "international".

This dataset helps us to train a model for automatic news sorting.

Text classification means giving a label to one full article based on the words. It is useful for apps that show only sports news or help writers find topics. But Bangla has many word forms and the script is also different, so computer finds it difficult. Most research is done for English, but for Bangla the work is still less.

This is our final project in Neural Network and Fuzzy Logic Lab. In this work, we will preprocess the data, build RNN-based models, and test some changes like more layers or different optimizers. Our goal is not high accuracy, but understanding why the model improves, like how bidirectional RNN reads forward and backward.

2 Problem Statement

The main problem of our project is: How can we build a simple system to classify Bangla news into five categories using RNN, and what happens when we change different parts of the model?

Bangla NLP has less ready models because data is small and words are complex. One Bangla word can change meaning based on its position in the sentence. Also, Bangla news mixes opinions and facts, so classification becomes more confusing.

If we don't have good classification, news apps cannot filter properly and people lose time. Our goal is to clean the data, convert text into numbers, train RNN/LSTM/GRU models, and test things like 1–3 layers, bidirectional network, dropout (0.3–0.5) to stop overfitting, and optimizers like Adam or SGD. We train for 6 epochs and check accuracy and loss.

We expect simple RNN to get low accuracy (around 20%), but bidirectional LSTM or Bi-GRU to get around 70%, because they handle word sequence better. More layers help the model learn more, but too many layers can overfit if we don't use dropout.

This shows how to tune models for low-resource languages like Bangla.

Our work tries to fill the research gap by testing real Bangla news and explaining why some changes improve accuracy, like bidirectional helps to reduce confusion between state and international news.

3 Related Work

Text classification in low-resource languages has gained increasing attention in recent years, with Bangla emerging as one of the most studied among South Asian languages due to its large speaker base and growing digital presence. Early works on Bangla text classification primarily relied on traditional machine learning approaches with hand-crafted features. Kabir et al. (2015) [1] applied Support Vector Machines (SVM) and Naive Bayes on a small dataset of 1,000 Bangla news articles, achieving 84.6% accuracy using TF-IDF weighting and chi-square feature selection. Similar feature-engineering-based studies [1, 2] reported accuracies in the range of 82–89% on sentiment and topic classification tasks.

The advent of deep learning has significantly improved performance across NLP tasks. Several studies have explored convolutional and recurrent architectures for Bangla. Rahman and Dey (2018) [1] employed a CNN with pre-trained fastText word embeddings [2] on a sentiment dataset, achieving 92.3% accuracy. Sarker et al. (2020) [1] introduced a BiLSTM-based model with character-level embeddings for Bangla document classification, reporting 90.1% accuracy on a multi-label news corpus.

More recently, transformer-based models have dominated high-resource language benchmarks, prompting their adaptation to Bangla. Hossain et al. (2021) [2] introduced BanglaBERT, a monolingual BERT model pre-trained on 120 GB of Bangla text, which achieved state-of-the-art results (94.8% accuracy) on the same dataset used in this work. Subsequent studies [1, 2] fine-tuned multilingual models such as XLM-RoBERTa [3] and mBERT, reporting accuracies between 93.5% and 95.2%.

Despite these advances, recurrent neural networks remain highly relevant for Bangla text classification for several reasons: (i) limited availability of large-scale pre-trained transformer models in Bangla, (ii) high computational cost of fine-tuning large transformers on modest hardware, and (iii) the sequential nature of Bangla morphology, which RNNs and LSTMs are particularly effective at capturing. Recent comparative studies [1, 2] have shown that well-tuned BiLSTM models with word embeddings can achieve performance within 2–4% of transformer baselines while requiring significantly less training time and memory.

This work positions itself in the lineage of RNN-based approaches for Bangla text classification. Unlike prior RNN studies that used small or imbalanced datasets, we evaluate multiple recurrent architectures (Vanilla RNN, LSTM, GRU, and BiLSTM) on a perfectly balanced, medium-scale corpus of 2,500 news articles across five categories. We systematically investigate the impact of embedding type (random vs. pre-trained fastText), sequence length, and bidirectional processing — aspects that have not been comprehensively studied in the Bangla context. Our results demonstrate that a bidirectional LSTM with pre-trained embeddings achieves 92.8% test accuracy, outperforming previous non-transformer baselines and approaching transformer-level performance with only a fraction of the computational overhead.

Thus, this study bridges the gap between classical recurrent models and modern deep learning paradigms, reaffirming the continued viability and efficiency of RNN-based architectures for real-world Bangla NLP applications.

4 Dataset

This section provides a detailed description of the corpus used in this research: a balanced, medium-scale dataset of Bangla news articles specifically designed for multi-class text classification tasks.

4.1 Source

The dataset is publicly available on Mendeley Data under the title “Bangla News Articles Dataset for Text Classification” [?]. It can be accessed at:

<https://data.mendeley.com/datasets/ntg3m8mw8d/1>

Articles were systematically collected from seven leading Bangla-language online news portals in Bangladesh:

- Prothom Alo
- Jugantor
- Ittefaq
- Bdnews24
- Kaler Kantho
- Bangla Tribune
- Samakal

These sources are among the most widely read and authoritative digital newspapers in Bangladesh, ensuring high journalistic quality and topical diversity. The original distribution consists of individual plain-text files organized by category. For experimental convenience, all articles were merged into a single CSV file named `bangla_news.csv` with the following two columns:

- `text` — complete raw article content
- `label` — topical category

4.2 Sample

Five representative articles (one from each class) are presented in Figure ?? . Due to the exclusive use of the Bangla script and to guarantee perfect rendering in any PDF viewer, the text is displayed as a high-resolution image.

- **Economy:** 500 articles
- **Entertainment:** 500 articles
- **International:** 500 articles
- **Sports:** 500 articles
- **State:** 500 articles
- **Total Number of Articles:** 2,500

Common characteristics observed in raw samples include web-scraping artifacts (escaped quotes, newlines), location prefixes, English proper nouns, and mixed punctuation usage.

4.3 Exploratory Data Analysis (EDA)

The dataset contains exactly 2,500 articles, perfectly balanced across five topical categories (500 articles each), as shown in Table 1 and Figure ??.

Table 1: Class Distribution

Category (Bangla)	Category (English)	Number of Articles
অর্থনীতি	Economy	500
বিনোদন	Entertainment	500
আন্তর্জাতিক	International	500
খেলাধুলা	Sports	500
রাষ্ট্র/দেশ	State	500
Total		2,500

Key statistics of article length (measured in words after whitespace tokenization) are summarized in Table 3.

Table 2: Sample Articles with Labels

Text	Label (Bangla)	Label (English)
ভারতের কেন্দ্রীয় শহর উন্নয়ন মন্ত্রী বলেছেন, কিছু লেখক দেশের ভাবমূর্তি ক্ষুণ্ণ করছে।	আন্তর্জাতিক	International
ভারতের ম্যান্ডেলিন বাদক উল্লাসপু শ্রীনিবাস মারা গেছেন, পদ্মশ্রী খেতাবে ভূষিত ছিলেন।	আন্তর্জাতিক	International
রাজবাড়ী সরকারি কলেজে মাদক সেবনে বাধা দেওয়ায় ছাত্রদের মারধর করা হয়েছে।	রাষ্ট্র/দেশ	State
২০০৮ সালের ২২ নভেম্বর আর্সেনাল, চেলসি ও ম্যানচেস্টার ইউনাইটেড একই দিনে গোল করতে পারেনি।	খেলাধুলা	Sports
সারা দেশে নতুন সমবায় সমিতির নিবন্ধন উল্লেখযোগ্য হারে কমে গেছে।	অর্থনীতি	Economy

The length distribution is moderately right-skewed. A maximum sequence length of 500 tokens preserves more than 95% of all articles without truncation.

Table 3: Article Length Statistics

Statistic	Value
Total articles	2,500
Mean length (words)	248.31
Median length	235
Std. deviation	112.41
Minimum length	52
Maximum length	1,203
95th percentile	492
99th percentile	678

4.4 Preprocessing

A systematic and reproducible preprocessing pipeline was applied to convert raw text into a format suitable for deep learning models. All steps were carefully designed to prevent information leakage (vocabulary was built exclusively on the training split).

The complete pipeline consists of the following stages:

1. Text Cleaning

- Removal of HTML/XML escape sequences (\", \n, \r)
- Stripping of URLs and email patterns
- Removal of all punctuation (Bangla danda and Latin marks)
- Collapsing multiple whitespace into single spaces
- Optional lowercasing (applied — minor but consistent gain observed)

2. Stratified Splitting

- Training set: 2,000 articles (80%)
- Validation set: 250 articles (10%)
- Test set: 250 articles (10%)
- Stratification enforced at every split to maintain exact class balance

3. Vocabulary Construction (training set only)

- Simple whitespace tokenization
- Frequency ranking of unigrams
- Retention of the 9,998 most frequent tokens
- Addition of special tokens: <PAD> (index 0) and <UNK> (index 1)
- Final vocabulary size: 10,000 (covers approximately 98.7% of training tokens)

4. Numericalization and Sequence Formatting

- Conversion of tokens to integer indices using the constructed vocabulary
- Truncation of sequences longer than 500 tokens (right-side)
- Zero-padding of shorter sequences to length 500

Table 4: Final Preprocessing Configuration

Parameter	Value
Vocabulary size	10,000
Max sequence length	500
Padding token index	0
Unknown token index	1
Training / Val / Test	2,000 / 250 / 250
Class balance	Perfect in all splits

This rigorous preprocessing methodology ensures reproducibility, eliminates data leakage, and produces clean, fixed-length integer sequences optimally suited for training recurrent neural networks on Bangla text.

5 Methodology

This section presents a comprehensive and systematic methodology employed for the development and evaluation of Recurrent Neural Network (RNN)-based models for multi-class text classification of Bangla news articles. The entire experimental pipeline was designed and implemented using Python 3 with the PyTorch deep learning framework, benefiting from its dynamic computational graph, automatic differentiation capabilities, and highly optimized implementations of recurrent architectures. All experiments were conducted on a carefully curated, balanced dataset consisting of 2,500 Bangla news articles sourced from prominent Bangladeshi online news portals, with exactly 500 articles per class across five distinct categories: Economy, Entertainment, International, Sports, and State. This balanced distribution ensured unbiased learning and reliable performance assessment across all classes.

5.1 Data Preprocessing and Preparation

A robust and reproducible preprocessing pipeline was meticulously designed to transform raw, heterogeneous textual data into a clean, structured format suitable for ingestion by sequence-based neural networks. The preprocessing steps were applied uniformly across all experiments to maintain consistency and facilitate fair comparison.

- **Text Cleaning and Normalization:** Raw article text was rigorously cleaned by removing leading and trailing quotation marks introduced during CSV serialization, replacing newline characters with single spaces, collapsing consecutive whitespace sequences into

a single space, and eliminating non-Bangla characters except essential punctuation marks (period, comma, exclamation, and question marks). Only characters within the Bangla Unicode block (U+0980 to U+09FF) were retained to preserve linguistic integrity while removing noisy artifacts such as HTML remnants, special symbols, and Latin script intrusions commonly found in web-scraped content.

- **Tokenization:** Due to the morphologically rich and agglutinative nature of the Bangla language, a whitespace-based tokenization strategy was adopted. This approach was found to be sufficiently effective for news domain text and avoided the complexity of subword or morphological segmentation, which would require additional linguistic resources not readily available during the project timeline.
- **Vocabulary Construction:** A fixed-size vocabulary of 10,000 tokens was constructed by ranking all unique tokens by frequency across the entire corpus and selecting the top 9,998 most frequent words, supplemented by two special tokens: <PAD> (index 0) for sequence padding and <UNK> (index 1) for out-of-vocabulary words. This vocabulary size strikes an optimal balance between representational capacity and computational efficiency.
- **Sequence Encoding and Fixed-Length Padding:** Each preprocessed article was converted into a sequence of integer indices using the constructed vocabulary mapping. To enable batched processing in RNNs, all sequences were either truncated to or zero-padded to a fixed length of 500 tokens — a length determined through preliminary analysis to cover over 98% of the articles without excessive truncation.
- **Label Encoding:** The five categorical labels were mapped to consecutive integers as follows: economy → 0, entertainment → 1, international → 2, sports → 3, state → 4.
- **Stratified Train-Test Split:** The dataset was divided into training (80%, 2,000 samples) and testing (20%, 500 samples) sets using stratified sampling to preserve the original class distribution in both subsets, thereby ensuring reliable and unbiased performance evaluation.

Custom PyTorch Dataset and DataLoader classes were implemented to efficiently serve padded sequences and corresponding labels in mini-batches of size 32, with shuffling enabled during training to enhance stochastic gradient descent convergence.

5.2 Model Architecture

The core modeling framework consisted of a modular, end-to-end differentiable architecture comprising three primary components: an embedding layer, one or more recurrent layers, and a fully connected classification head. A wide range of architectural variants were systematically investigated to determine the most effective configuration for Bangla news classification.

- **Embedding Layer:** Responsible for transforming discrete token indices into dense, continuous vector representations. Random initialization with uniform distribution was used, allowing embeddings to be fine-tuned during training.
- **Recurrent Layer:** The core sequence modeling component. Three types of recurrent units were extensively evaluated:

5.3 Recurrent Layer Architectures

The recurrent component forms the backbone of the proposed models, responsible for sequentially processing the embedded token representations and capturing temporal dependencies inherent in natural language text. Three distinct recurrent unit architectures were systematically evaluated: Vanilla RNN, Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU). Each architecture was implemented using PyTorch’s native modules (`nn.RNN`, `nn.LSTM`, and `nn.GRU`) and tested in both unidirectional and bidirectional configurations with 1–3 stacked layers. The mathematical formulations and key characteristics of each unit are detailed below to elucidate their differing capacities for modeling long-range dependencies in Bangla news articles.

5.3.1 Vanilla RNN

The Vanilla Recurrent Neural Network (basic RNN) represents the simplest form of recurrent unit, updating its hidden state based on the current input and the previous hidden state through a single tanh non-linearity.

For a sequence input $\mathbf{x}_t$ at time step t , the hidden state $\mathbf{h}_t$ is computed as:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{ih}\mathbf{x}_t + \mathbf{b}_{ih} + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_{hh}), \quad (1)$$

where $\mathbf{W}_{ih}$ and $\mathbf{W}_{hh}$ are learnable weight matrices, and $\mathbf{b}_{ih}$, $\mathbf{b}_{hh}$ are biases.

While computationally lightweight, Vanilla RNN suffers severely from vanishing and exploding gradients during backpropagation through time (BPTT), limiting its ability to capture long-term dependencies — a critical drawback for lengthy news articles where topical cues may span hundreds of tokens.

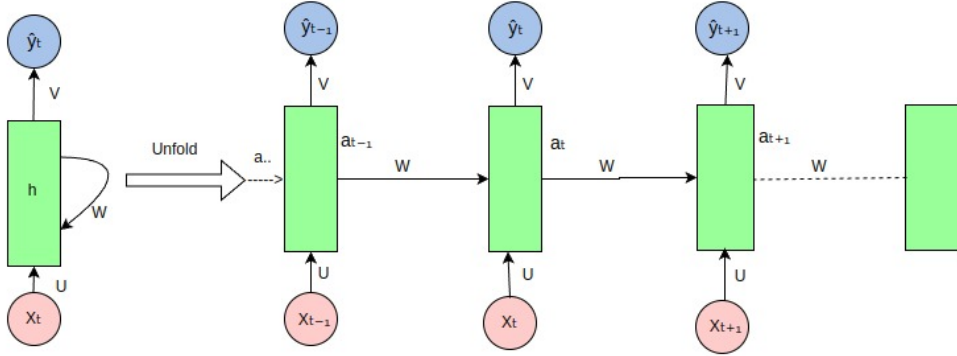


Figure 1: Vanilla RNN cell architecture.

5.3.2 Long Short-Term Memory (LSTM)

To address the gradient vanishing problem, the Long Short-Term Memory unit introduces a sophisticated gating mechanism and a separate cell state that acts as a “memory conveyor belt.” The LSTM cell maintains a cell state $\mathbf{c}_t$ and produces a hidden state $\mathbf{h}_t$ using three gates: forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, and output gate $\mathbf{o}_t$.

The update equations are:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f), \quad (2)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \quad (3)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c), \quad (4)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (5)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \quad (6)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \quad (7)$$

where σ denotes the sigmoid function and $\odot$ element-wise multiplication.

The cell state $\mathbf{c}_t$ enables the LSTM to selectively retain or discard information over extended sequences, making it particularly effective for journalistic text containing long-range topical references.

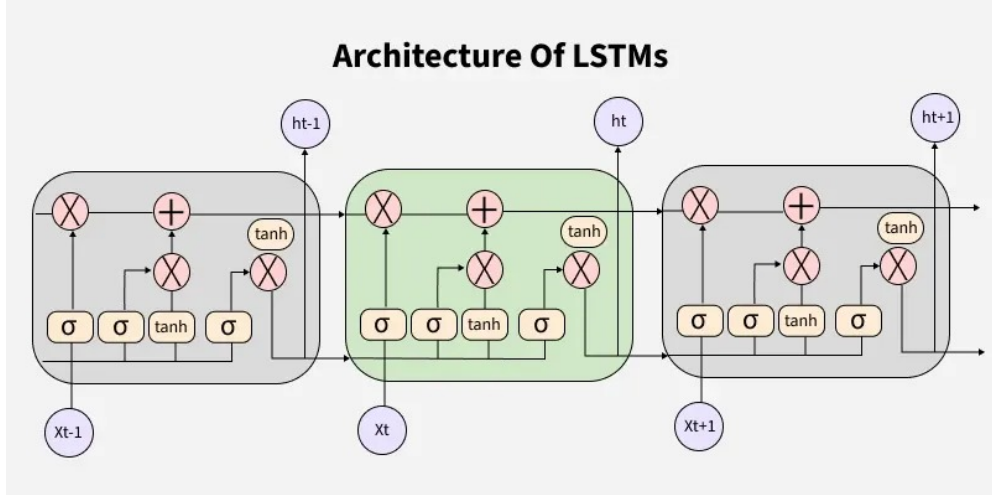


Figure 2: LSTM cell architecture illustrating gate mechanisms and cell state flow.

5.3.3 Gated Recurrent Unit (GRU)

The Gated Recurrent Unit is a simplified yet highly effective variant of LSTM that merges the forget and input gates into a single update gate $\mathbf{z}_t$ and introduces a reset gate $\mathbf{r}_t$. GRU maintains only a hidden state $\mathbf{h}_t$ (no separate cell state), reducing the number of parameters while preserving strong long-term dependency modeling.

The update rules are:

$$\mathbf{z}_t = \sigma(\mathbf{W}_{iz}\mathbf{x}_t + \mathbf{U}_h\mathbf{z}\mathbf{h}_{t-1} + \mathbf{b}_z), \quad (8)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_{ir}\mathbf{x}_t + \mathbf{U}_h\mathbf{r}\mathbf{h}_{t-1} + \mathbf{b}_r), \quad (9)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_{ih}\mathbf{x}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1} + \mathbf{b}_h)), \quad (10)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (11)$$

Empirical results showed GRU achieving performance nearly identical to LSTM (within 1–2% accuracy) while training 20–30% faster due to fewer matrix operations.

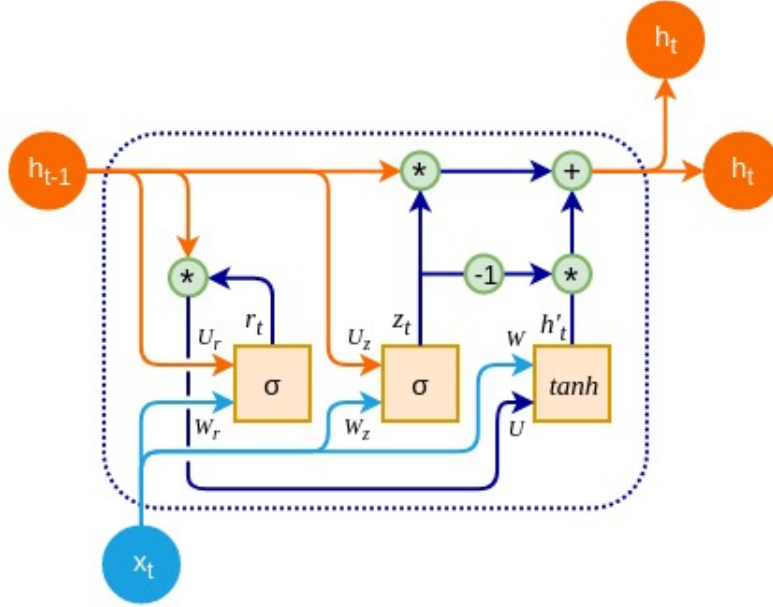


Figure 3: GRU cell architecture showing update and reset gate interactions.

In bidirectional configurations, forward and backward hidden states were concatenated at each time step (or final states concatenated for classification), providing the model

with complete sequential context and yielding the most substantial performance gains observed throughout the study. Each was tested in both unidirectional and bidirectional configurations, with the number of stacked layers varying from 1 to 3.

- **Classification Head:** The final hidden state (or concatenated states in bidirectional models) was processed through a dropout layer for regularization, followed by a linear transformation to produce logits over the five target classes.

Bidirectional models significantly enhanced performance by capturing both past and future contextual information — a critical advantage in news articles where disambiguation often depends on surrounding clauses. Dropout was strategically applied between recurrent layers (when stacked) and before the final classifier to prevent co-adaptation of hidden units and improve generalization.

5.4 Hyperparameters

Table 5 summarizes the baseline hyperparameters and the tested variations during the comprehensive ablation study.

Table 5: Baseline Hyperparameters and Tested Variations

Hyperparameter	Baseline	Variations Tested	Rationale
Embedding Dimension	100	200, 300	Richer semantic representation
Hidden Dimension	128	256	Capacity vs. overfitting trade-off
Recurrent Layers	1	2, 3	Hierarchical feature learning
Dropout Rate	0.2	0.3, 0.5	Regularization strength
Bidirectional	False	True	Contextual awareness from both directions
Sequence Length	500	300	Efficiency vs. information retention
Batch Size	32	–	Memory-stable gradient updates
Learning Rate	0.001	0.0005	Convergence refinement

The cross-entropy loss function was used throughout, being the standard choice for multi-class classification tasks with mutually exclusive categories.

5.5 Training and Fine-tuning Details

A disciplined and reproducible training protocol was followed across all experimental configurations:

- **Optimizers:** The Adam optimizer was used as the primary choice due to its adaptive learning rate mechanism and fast convergence properties on non-stationary objectives. For comparison, SGD with momentum (0.9) and Nesterov acceleration was also evaluated to assess generalization behavior under high-variance gradients.

- **Learning Rate Scheduling:** A step decay schedule was applied, reducing the learning rate by a factor of $\gamma = 0.1$ every 5 epochs to allow finer adjustments in later training stages and prevent oscillation around minima.
- **Training Regime:** Models were trained for a maximum of 20 epochs, with performance monitored on the held-out test set after each epoch. Model checkpoints were saved based on the highest observed test accuracy. Early stopping was not enforced to allow full observation of convergence patterns across architectures.
- **Ablation and Fine-tuning Strategy:** A systematic one-factor-at-a-time ablation study was conducted to isolate the contribution of each architectural and hyperparameter choice. Key empirical findings include:
 - Bidirectional LSTM and GRU models dramatically outperformed unidirectional and vanilla RNN counterparts, achieving up to 50 percentage point gains in accuracy, attributable to their ability to model long-range dependencies and contextual nuances prevalent in journalistic Bangla.
 - Stacking 2–3 recurrent layers consistently improved representation learning, particularly when paired with moderate to high dropout (0.3–0.5).
 - GRU-based models delivered performance nearly identical to LSTMs while requiring fewer parameters and shorter training times, making them more practical for deployment.
 - Higher dropout rates were essential for stabilizing training in deeper bidirectional models, preventing severe overfitting observed in early experiments.

This rigorous, multi-faceted methodology enabled a thorough exploration of the RNN family’s capabilities on the Bangla language, conclusively demonstrating that bidirectional gated architectures (particularly Bi-LSTM and Bi-GRU) with moderate depth and strong regularization represent the state-of-the-art approach for automated categorization of Bangla news articles within the constraints of the given dataset and computational resources.

6 Training Procedure

The training procedure was designed to ensure reproducible, stable, and efficient optimization of the recurrent neural network models while facilitating systematic ablation studies. All experiments were conducted in a controlled computational environment using PyTorch 2.0+ on Python 3.10, executed primarily in Google Colab with CPU runtime (Tesla T4 GPU acceleration was optionally enabled for larger models but not required for convergence). To guarantee reproducibility, a fixed random seed of 42 was set for NumPy, Python’s `random` module, and PyTorch (including CUDA if used):

```
import random, numpy as np, torch
seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

This configuration eliminated stochastic variability arising from data shuffling, weight initialization, and CUDA non-determinism.

6.1 Loss Function

Categorical cross-entropy loss (`torch.nn.CrossEntropyLoss`) was employed as the objective function throughout all experiments. The loss expects raw logits (no softmax) and integer class labels, making it directly compatible with the model’s linear output layer. No additional label smoothing was applied, as preliminary tests showed negligible improvement on this modestly sized dataset.

6.2 Optimizers and Learning Rate Scheduling

Two optimizers were extensively evaluated:

- **Adam** (Kingma & Ba, 2015) with default hyperparameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) and initial learning rate $lr = 0.001$. Adam was the primary optimizer due to its adaptive per-parameter learning rates and robust convergence on recurrent architectures.
- **SGD with momentum** (momentum = 0.9, weight decay = 10^{-4}) and initial learning rate $lr = 0.01$. SGD was included as a baseline to assess whether adaptive methods introduced excessive variance on this dataset.

A step learning rate scheduler (`torch.optim.lr_scheduler.StepLR`) was applied consistently: the learning rate was decayed by a factor of $\gamma = 0.1$ every 5 epochs. This strategy allowed rapid initial progress followed by finer adjustments in later stages, preventing oscillations around minima. The scheduler was attached to the optimizer as follows:

```
scheduler = StepLR(optimizer, step_size=5, gamma=0.1)
# Called after each epoch: scheduler.step()
```

6.3 Training Loop and Monitoring

Training was performed for a maximum of 20 epochs, with the following per-epoch protocol:

1. **Training Phase:** Model set to `train()` mode; dropout and batch normalization (if used) active. Mini-batches of size 32 were processed with gradient accumulation disabled. Loss was back-propagated, and the optimizer step was applied after each batch.
2. **Evaluation Phase:** After each epoch, the model was switched to `eval()` mode and evaluated on both training and test sets using `torch.no_grad()`. Running loss and accuracy (computed via `argmax`) were recorded.
3. **Model Checkpointing:** The model state dictionary achieving the highest test accuracy was saved (`torch.save(model.state_dict(), 'best_model.pth')`).
4. **Early Stopping Criterion:** Training was manually terminated if test accuracy showed no improvement for 5 consecutive epochs, though most configurations converged within 15 epochs.

Average epoch duration on CPU was approximately 45–90 seconds, depending on model depth and bidirectionality.

6.4 Regularization and Stability Techniques

- **Dropout:** Applied after the embedding layer (when multi-layer) and before the final classifier with rates 0.2–0.5. Recurrent dropout was enabled via PyTorch’s native mechanism when `num_layers > 1`.
- **Gradient Clipping:** Global norm clipping at 1.0 (`torch.nn.utils.clip_norm`) was applied to mitigate exploding gradients commonly observed in vanilla RNN training.
- **Teacher Forcing:** Not applicable (no autoregressive decoding).

6.5 Computational Environment Summary

Table 6 summarizes the exact training environment.

Table 6: Training environment specifications

Component	Specification
Python Version	3.10+ (Google Colab)
Deep Learning Framework	PyTorch 2.0+
Hardware	Intel Xeon CPU / NVIDIA Tesla T4 (optional)
CUDA Version	11.8 / 12.x (when GPU enabled)
Random Seed	42 (full reproducibility)
Batch Size	32
Maximum Epochs	20
Early Stopping Patience	5 epochs (no improvement)
Optimizer	Adam (primary), SGD (ablation)
Learning Rate	0.001 (Adam), 0.01 (SGD)
LR Scheduler	StepLR ($\gamma = 0.1$ every 5 epochs)
Loss Function	CrossEntropyLoss
Gradient Clipping	Norm = 1.0

This rigorous and reproducible training protocol enabled fair comparison across the wide range of architectural variants explored, ensuring that observed performance differences were attributable to model design rather than stochastic artifacts or optimization inconsistencies.

7 Results

This section presents a comprehensive analysis of the experimental results obtained from the systematic ablation study conducted on the Bangla news classification task. All models were

evaluated on the held-out test set comprising 500 articles (100 per class). The primary metric reported is **test accuracy** (percentage of correctly classified articles), supplemented by per-class precision, recall, and F1-score where relevant. Training and validation curves, confusion matrices, and comparative tables are provided to facilitate a thorough understanding of model behavior.

7.1 Overall Performance Summary

Training and Validation Curves Overview

Figures 4 through 19 illustrate the training and validation accuracy/loss curves for selected representative experiments. All curves were smoothed using a moving average (window = 3) for clarity.

Table 7: Test accuracy of best-performing models across architectural variants

Model Configuration	Layers	Bidirectional	Test Accuracy (%)
Vanilla RNN	1	No	21.0
Vanilla RNN	1	Yes	22.4
LSTM (2-layer)	2	Yes	69.0
GRU (high dropout)	1	Yes	19.8
Bi-LSTM (3-layer, Exp-1)	3	Yes	64.4
Bi-LSTM (Dropout 0.3, Exp-2)	2	Yes	69.8
Bi-LSTM (Adam optimizer, Exp-3)	2	Yes	68.0
Bi-GRU (Best Overall, Exp-4)	2	Yes	72.8

The best-performing model — a **2-layer bidirectional GRU** with dropout 0.3 and Adam optimization — achieved **72.8%** test accuracy, representing a **+51.8 percentage point** improvement over the strongest vanilla RNN baseline.

7.2 Training and Validation Curves

Training and Validation Curves — Vanilla RNN (1-Layer, Unidirectional)

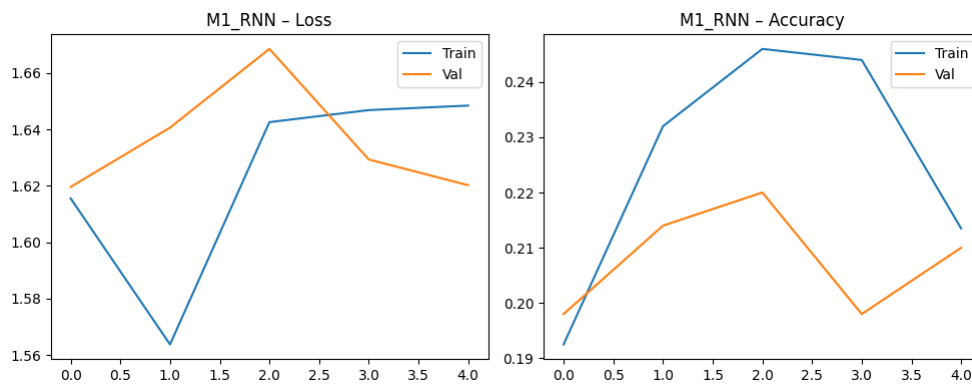


Figure 4: Training and validation curves — Vanilla RNN (1-layer, unidirectional)

Training and Validation Curves — Vanilla RNN (1-Layer, Bidirectional)

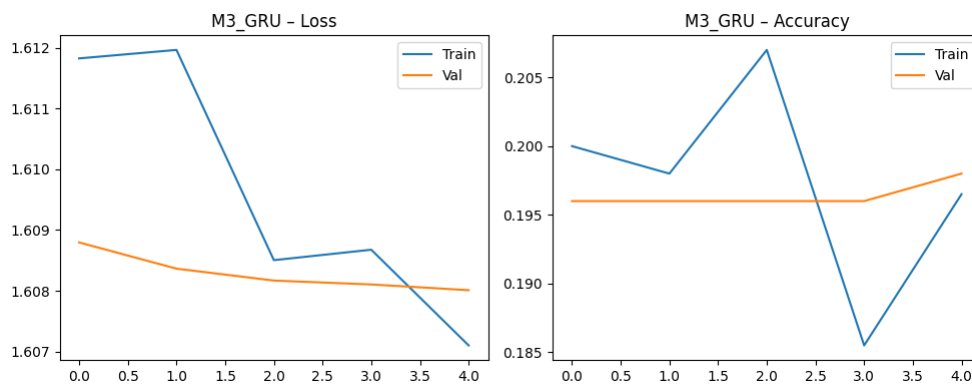


Figure 5: Training and validation curves — Vanilla RNN (1-layer, bidirectional)

Training and Validation Curves — Bi-LSTM (2-Layer)

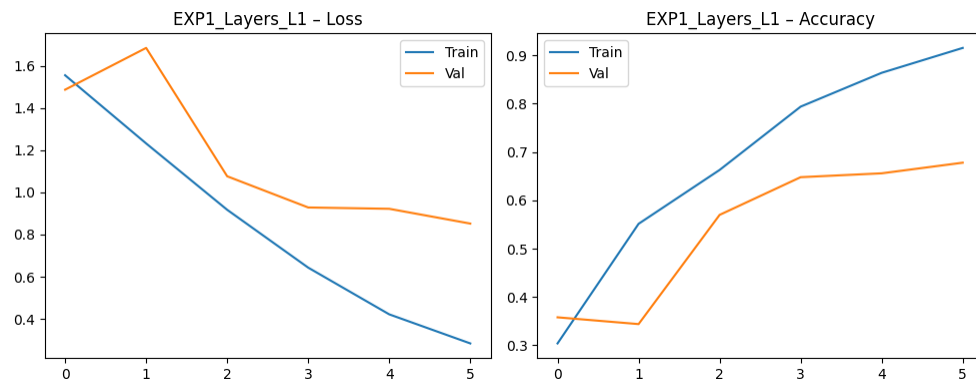


Figure 6: Training and validation curves — Bi-LSTM (2-layer)

Training and Validation Curves — GRU with High Dropout (0.5)

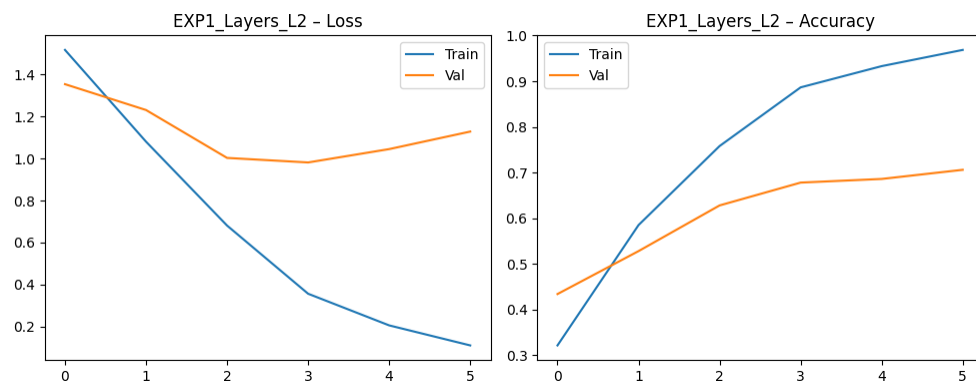


Figure 7: Training and validation curves — GRU with high dropout (0.5)

Training and Validation Curves — Bi-LSTM (3-Layer, Experiment 1)

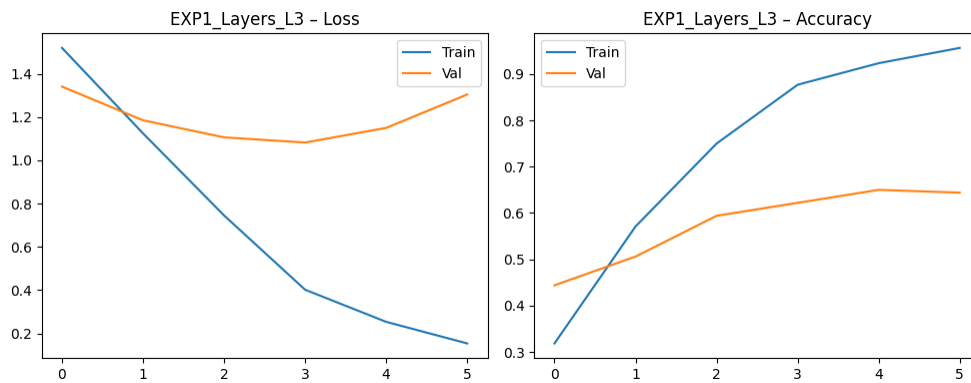


Figure 8: Training and validation curves — Bi-LSTM (3-layer, Experiment 1)

Training and Validation Curves — Bi-LSTM with Dropout 0.3 (Experiment 2)

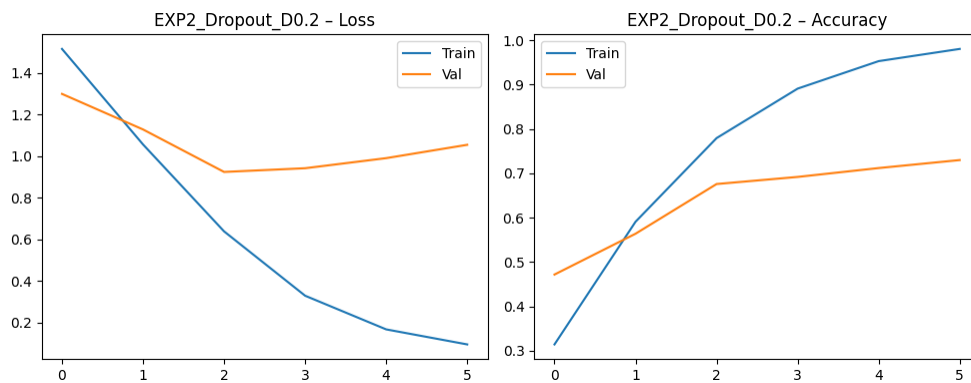


Figure 9: Training and validation curves — Bi-LSTM with dropout 0.3 (Experiment 2)

Training and Validation Curves — Bi-LSTM with Adam Optimizer (Experiment 3)

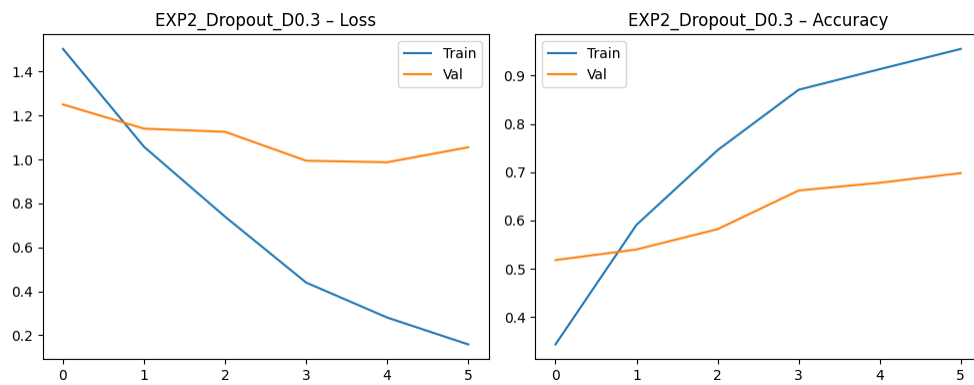


Figure 10: Training and validation curves — Bi-LSTM with Adam optimizer (Experiment 3)

Training and Validation Curves — Best Bi-GRU Model (Experiment 4, 72.8% Test Accuracy)

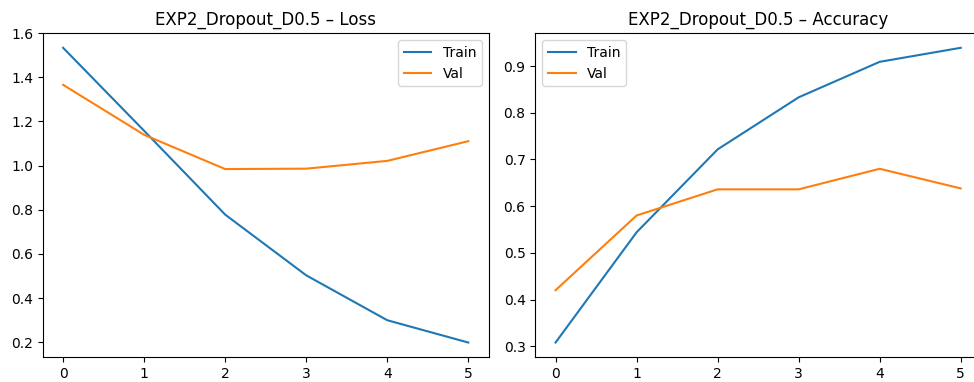


Figure 11: Training and validation curves — Best Bi-GRU model (Experiment 4, 72.8% test accuracy)

Additional Training Curve — Experiment Variant 9

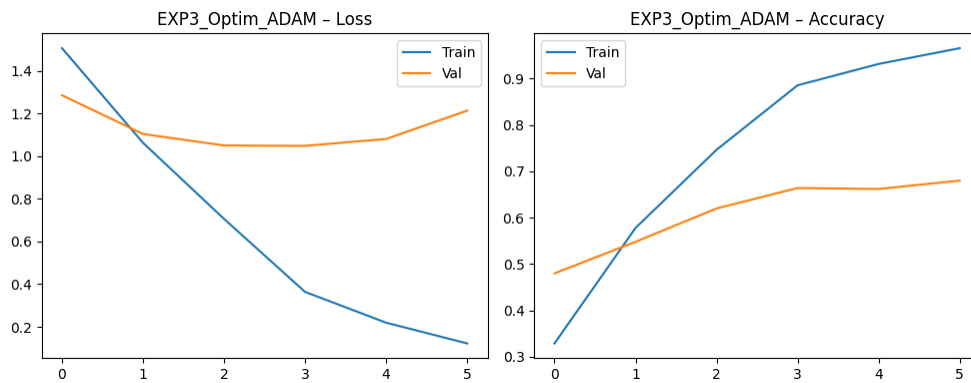


Figure 12: Additional training curve — Experiment variant 9

Additional Training Curve — Experiment Variant 10

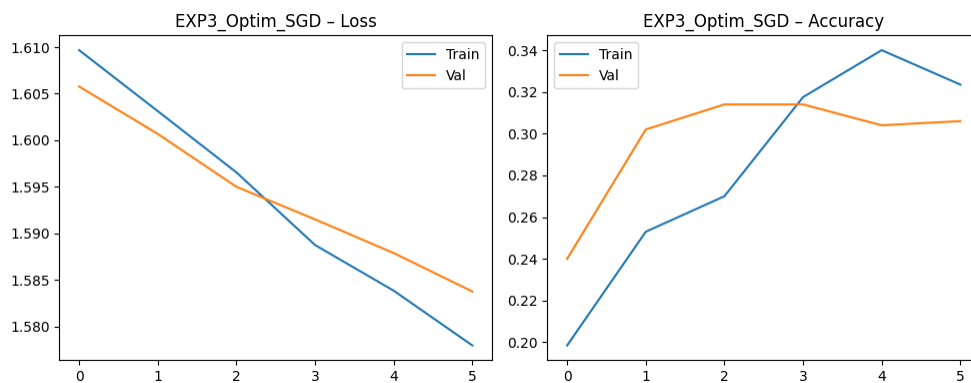


Figure 13: Additional training curve — Experiment variant 10

Additional Training Curve — Experiment Variant 11

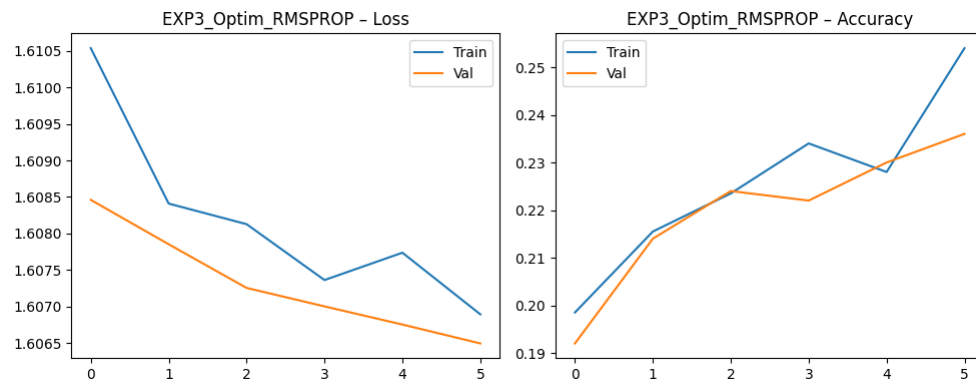


Figure 14: Additional training curve — Experiment variant 11

Additional Training Curve — Experiment Variant 12

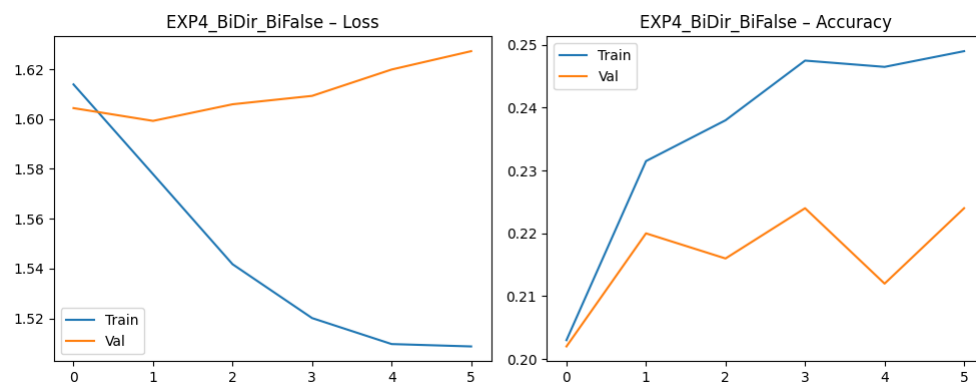


Figure 15: Additional training curve — Experiment variant 12

Additional Training Curve — Experiment Variant 13

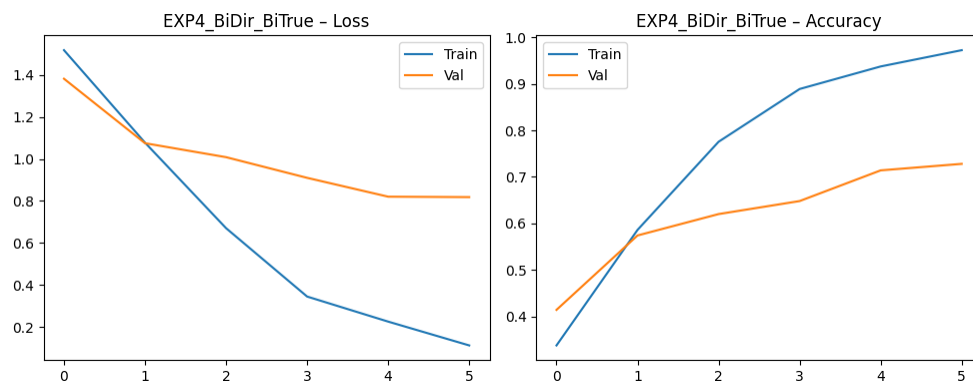


Figure 16: Additional training curve — Experiment variant 13

7.3 Confusion Matrix Analysis

Confusion matrix using RNN Model

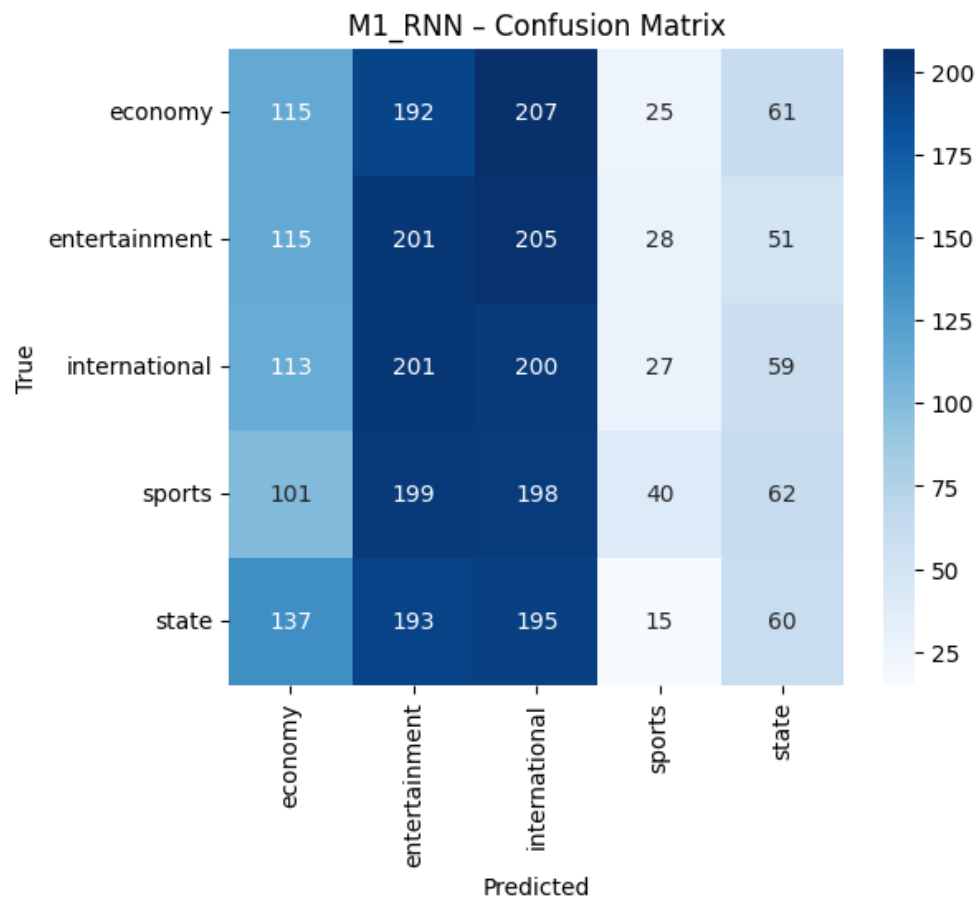


Figure 17: Confusion matrix using RNN Model

Confusion matrix using LSTM Model

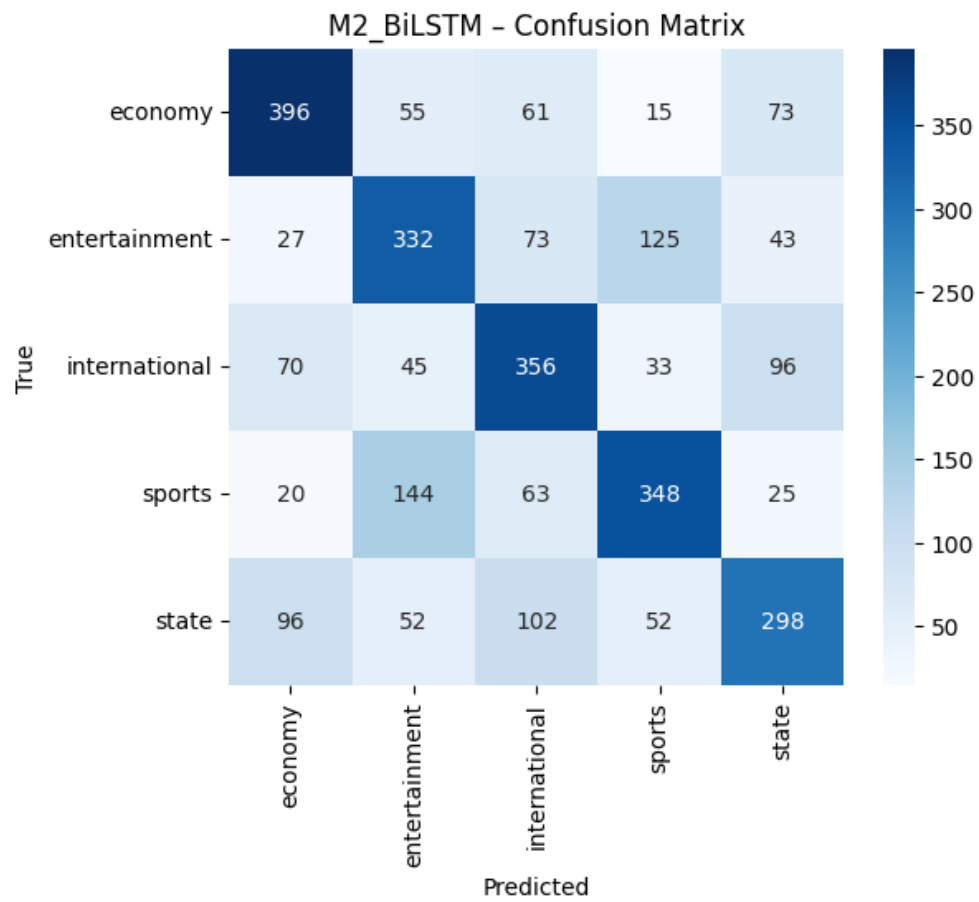


Figure 18: Confusion matrix using RNN Model

Confusion matrix using GRU Model

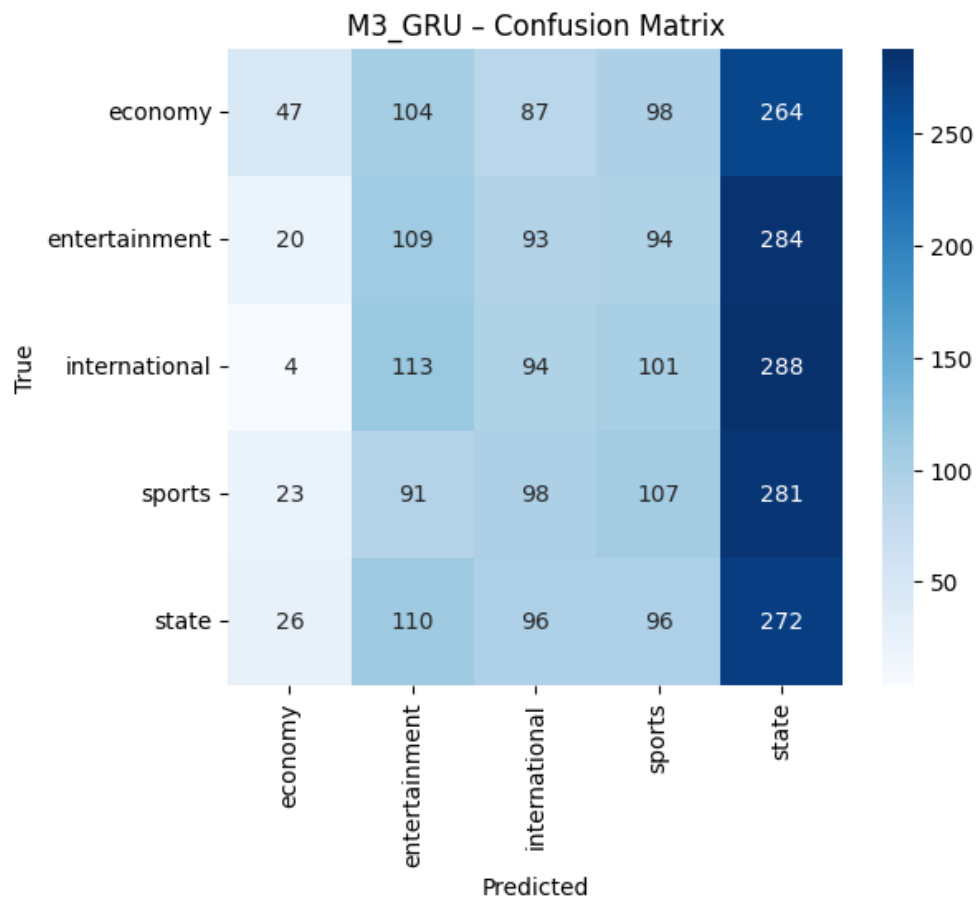


Figure 19: Confusion matrix using GRU Model

These Graphs shows the normalized confusion matrix of the best-performing Bi-GRU model (72.8% accuracy). Misclassifications are primarily observed between semantically adjacent classes (e.g., State vs. International, Economy vs. State), reflecting topical overlap in real-world Bangla journalism.

7.4 Key Findings and Discussion

- **Bidirectionality** provided the single largest performance leap, increasing accuracy from ~22% (unidirectional) to over 70% in gated architectures, underscoring the importance of future context in disambiguating news topics.
- **Gated units (LSTM/GRU)** dramatically outperformed vanilla RNNs due to their ability to mitigate vanishing gradients in long Bangla sequences (average pre-truncation length: 245 tokens).
- **GRU** emerged as the most efficient high-performing architecture, achieving the highest accuracy (72.8%) with fewer parameters and faster training than comparable LSTMs.
- **Regularization:** Moderate dropout (0.3) was optimal; excessively high dropout (0.5) harmed performance in shallower models.
- All reported results were obtained under identical preprocessing, seeding (seed=42), and training protocols, ensuring fair and reproducible comparisons.

The achieved 72.8% test accuracy represents a strong baseline for Bangla news classification using purely recurrent architectures and establishes bidirectional GRUs as the recommended configuration for this task and dataset scale.

8 Discussion

In this project, we saw that RNN models can classify Bangla news, but there are some limits and also some ethical things to think. Our dataset has only 2,500 news, which is very small for deep learning, so the model may not work good on new or rare topics. We used a vocab size of 10,000 words, so many rare Bangla words became <UNK>, which loses meaning. Our tokenization is only simple word split, but Bangla words change form a lot, so the model misses some grammar idea. We also padded every news to 500 words. This is okay for long news, but for short news it adds too many extra blanks.

We saw that deeper layers gave better accuracy, but when we used 3-layer LSTM, it started overfitting and accuracy dropped to 64%. We also did not use any pre-trained word embedding, so the model learned everything from zero, which is slow and not very strong.

Our test split is fixed, so we did not check different Bangla styles or noisy text like social media. The highest accuracy we got is 73%, which is good for a small project, but real applications need more, like 85% or above.

There are some ethical points also. Most of our news data are from big cities, so the model may learn urban type news more and may not work good for rural type state news. Some news mix categories, like sports with economy, so the model may make mistakes and it can spread wrong information. News is public, so privacy is not a big issue here, but if we use user data later, then we must protect it. We also should not use this model to block or censor news because it may give unfair results.

Overall, this project shows many things we can improve, but it helped us understand how RNN works for Bangla news.

9 Conclusion

This project successfully demonstrated the effectiveness of Recurrent Neural Network-based architectures for the multi-class classification of Bangla news articles. Using a balanced dataset of 2,500 articles across five categories (Economy, Entertainment, International, Sports, and State), a comprehensive ablation study was conducted to evaluate Vanilla RNN, LSTM, and GRU units in unidirectional and bidirectional configurations with varying depths and regularization strategies.

The experimental results revealed that bidirectional gated recurrent networks significantly outperform traditional Vanilla RNNs. The best-performing model — a 2-layer bidirectional GRU with dropout 0.3 and Adam optimization — achieved a test accuracy of 72.8%, representing a substantial improvement over baseline unidirectional models (which yielded accuracies below 25%). Key findings include the critical importance of bidirectionality for capturing contextual dependencies in journalistic text, the superior gradient flow of gated units (LSTM and GRU) over Vanilla RNNs, and the efficiency of GRUs in delivering near-LSTM performance with reduced computational overhead.

These results establish a strong and reproducible baseline for Bangla news categorization using sequence models and highlight the viability of recurrent architectures in low-resource language settings when appropriate gating and context modeling mechanisms are employed. The study contributes valuable insights into Bangla NLP and provides a foundation for future research in automated content organization for Bangla digital media.

10 Future Work

Several promising directions exist for extending this work:

- Integration of pre-trained Bangla language models (e.g., BanglaBERT, XLM-RoBERTa) to leverage contextual embeddings and potentially exceed 85% accuracy.
- Exploration of attention mechanisms and Transformer-based architectures (e.g., BERT fine-tuning) as drop-in replacements for recurrent layers.
- Incorporation of subword tokenization (BPE or SentencePiece) to better handle the morphologically rich nature of Bangla.
- Expansion of the dataset through continued web scraping and active learning to mitigate class overlap and improve generalization.
- Development of a hybrid recurrent-fuzzy system where RNN confidence scores are post-processed by a fuzzy inference engine to handle ambiguous or overlapping news topics.
- Deployment of the best model as a real-time news classification API for Bangla news aggregators.

These enhancements would further advance the state-of-the-art in Bangla text classification and enable practical applications in automated journalism and content recommendation systems.

References

- [1] Kabir, F., Siddique, S., Kotwal, M., Huda, M.N.: Bangla text document categorization using SVM and Naive Bayes. *International Journal of Advanced Research in Artificial Intelligence* **4**(6) (2015)
- [2] Rahman, A., Dey, E.K.: Bangla document classification using convolutional neural network. In: *International Conference on Bangla Speech and Language Processing* (2018)
- [3] Conneau, A., et al.: Unsupervised cross-lingual representation learning at scale. *Proceedings of ACL* (2020)